Database Connection Pooling Guide

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

This guide is how we manage database and Spanner client connections so a service does not exhaust the backend under load. Two failures recur: opening a fresh Postgres connection per request until the server refuses new clients, and holding Spanner sessions until the client's session pool runs dry. Both come from the same root cause — unbounded or unreturned connections. The rules below bound every pool, always release connections, and keep total connections safely below the server ceiling. Applies to api-service, worker-service, ingestion-service, and scheduler.

Standards

Use one bounded pool per process

Never open a connection per request. Create a single SQLAlchemy engine at startup with an explicit, bounded pool and reuse it. Size `pool_size + max_overflow` so that (instances × per-instance total) stays under the server's `max_connections`, leaving headroom for migrations and admin.

```
from sqlalchemy import create_engine

engine = create_engine(
   settings.database_url,
   pool_size=10,          # steady-state connections held
   max_overflow=5,        # burst above pool_size
   pool_timeout=30,       # wait for a free conn, then fail fast
   pool_recycle=1800,     # recycle before server/idle timeout
   pool_pre_ping=True,    # detect dropped connections
)
```

Budget capacity explicitly: if Postgres allows 100 connections and you run up to 6 instances, cap each instance at ≤15 total (`pool_size=10`, `max_overflow=5`) so the fleet peaks near 90, not 100+.

Always return connections to the pool

A leaked connection is never reused, so the pool drains and requests block on `pool_timeout`. Use context managers so connections and sessions are released even on exceptions.

```
with engine.connect() as conn:          # returned to pool on exit
   conn.execute(stmt)

# ORM
with Session(engine) as session:
   session.add(obj)
   session.commit()
```

For very high connection counts or many short-lived clients, put PgBouncer in transaction-pooling mode in front of Postgres so the app pool multiplexes onto far fewer server connections.

Size the Spanner session pool and release sessions

The Spanner client keeps its own session pool. Under load, if sessions are held across slow work they run out and requests fail with “no sessions available.” Size the pool for peak concurrency and always release sessions with context managers.

```
from google.cloud.spanner_v1 import Client
from google.cloud.spanner_v1.pool import FixedSizePool

client = Client()
instance = client.instance("app-instance")
db = instance.database("app-db", pool=FixedSizePool(size=50))   # >= peak concurrency

with db.snapshot() as snapshot:              # session released on exit
   rows = snapshot.execute_sql("SELECT ...")
```

- Never hold a session (or DB connection) across a slow external call — acquire late, release early.
- Do not open transactions you leave idle; keep them short.

Common failures and fixes

- Postgres `FATAL: sorry, too many clients already` because each request opens a new connection → use one bounded engine (`pool_size`/`max_overflow`) or PgBouncer, return connections to the pool, and cap total connections below the server limit (BUG-1004).
- Spanner requests block or fail with “no sessions available” under load → size the session pool for peak concurrency, use context managers so sessions are released, and don't hold sessions across slow external calls (BUG-1030).
- Requests block then time out on `pool_timeout` → a connection leak; wrap every acquisition in a context manager and check for missing `commit`/`close`.

Checklist

- One engine/client per process, created at startup and reused — no per-request connections.
- Pool is explicitly bounded (`pool_size`, `max_overflow`, `pool_timeout`); Spanner uses a sized `FixedSizePool`.
- Fleet math verified: instances × per-instance total < server `max_connections`, with headroom.
- Every connection/session is acquired via a context manager and released on error paths.
- `pool_recycle`/`pool_pre_ping` set to survive idle-timeout drops; PgBouncer used where connection counts are high.
- No connection or session is held across a slow external call.

Related incidents

BUG-1004, BUG-1030